

정렬 알고리즘 비교 보고서

과목: 2026-2 고급알고리즘(SIT2001-01) — 과제1 저장소: <https://github.com/silvpear654/algorithms-hw1>

1. 개요

이 보고서는 세 가지 정렬 알고리즘을 비교한다.

구분	알고리즘	비고
배운 것 1	버블 정렬 (Bubble Sort)	저장소 템플릿에 있던 예제
배운 것 2	퀵 정렬 (Quick Sort)	이번 과제에서 추가
안 배운 것	블록 정렬 (Block Sort)	이번 과제에서 추가, AI로 학습

안 배운 것으로 블록 정렬을 고른 이유는, 수업에서 배운 병합 정렬과 같은 "나눠서 정렬한 뒤 합친다"는 뼈대를 공유하면서도, 병합 정렬의 가장 큰 단점인 "병합할 때 크기 n 짜리 보조 배열이 필요하다"는 문제를 다른 방식으로 해결하기 때문이다. 배운 알고리즘의 한계를 어떻게 극복하는지 볼 수 있어 비교하기 좋다고 판단했다.

세 알고리즘 모두 C(`bubbleSort` / `quickSort` / `blockSort` , `src/sort.c`)와 Python(`bubble_sort` / `quick_sort` / `block_sort` , `src/sort.py`)으로 동일하게 구현했고, 같은 이름의 함수로 같은 알고리즘을 구현한다는 저장소 규약을 따랐다. 실행과 성능 비교는 `src/main.c` / `src/main.py` 에서 한다. 정확성 테스트는 `tests/test_sort.c` , `tests/test_sort.py` 에서 세 알고리즘에 대해 동일한 케이스(섞인 배열, 정렬된 배열, 역순, 중복, 원소 하나, 빈 배열, 그리고 블록 정렬의 병합 경로를 실제로 거치도록 만든 200개짜리 큰 배열)를 검증한다.

2. 알고리즘 설명

2.1 버블 정렬 (배움)

인접한 두 원소를 비교해 순서가 틀리면 교환하는 과정을 배열 끝까지 반복하고, 이를 배열 전체에 대해 반복한다. 한 바퀴 돌 때마다 가장 큰 값이 뒤로 '거품처럼' 떠오른다. 한 바퀴 동안 교환이 한 번도 없었다면 이미 정렬된 것이므로 조기 종료한다.

- 시간 복잡도: 최선 $O(n)$ (이미 정렬된 경우, 조기 종료), 평균/최악 $O(n^2)$
- 공간 복잡도: $O(1)$ (제자리 정렬)
- 안정 정렬

2.2 퀵 정렬 (배움)

배열에서 기준값(pivot)을 하나 고르고, 기준값보다 작은 값은 왼쪽, 큰 값은 오른쪽으로 모은 뒤(분할, partition), 양쪽을 각각 재귀적으로 정렬한다. 구현에는 Lomuto 분할을 사용했고, 배열의 마지막 원소를 pivot으로 삼는다. 분할 후 더 작은 쪽만 재귀 호출하고 큰 쪽은 반복문으로 처리해(`src/sort.c` , `src/sort.py`) 재귀 깊이를 최악의 경우에도 $O(\log n)$ 으로 묶었다.

- 시간 복잡도: 평균 $O(n \log n)$, 최악 $O(n^2)$ (이미 정렬되었거나 pivot이 계속 한쪽으로 치우치는 경우)
- 공간 복잡도: $O(\log n)$ (재귀 스택), 제자리 정렬
- 불안정 정렬

2.3 블록 정렬 (위키피디아)

핵심 문제의식: 병합 정렬은 $O(n \log n)$ 을 보장하지만, 두 정렬된 구간을 합칠 때 보통 크기 n 짜리 보조 배열을 새로 만든다 ($O(n)$ 추가 공간). 블록 정렬은 "어떻게 하면 보조 배열 없이($O(1)$ 공간) 병합할 수 있는가"라는 문제를 푸는 알고리즘 계열이다. 위키피디아의 정렬 알고리즘 비교표도 블록 정렬을 "블록 기반 접근과 제자리(in-place) 병합 방식을 결합한 것 (Combine a block-based approach with the in-place method for merging)"이라고 설명한다. 실제 프로덕션급 구현 (WikiSort, Grailsort 등)은 여기에 내부 버퍼·태그 배열 같은 정교한 장치를 더 쓰지만, 이번 구현은 그 핵심 아이디어만 추린 간소화 버전이다.

구현은 두 단계로 이루어진다.

1단계 — 블록 단위 삽입 정렬. 배열을 크기 32(상수)짜리 블록으로 나누고, 각 블록을 삽입 정렬로 정렬한다 (`src/sort.c` , `src/sort.py`). 블록 크기를 n 에 비례하게(예: $\sqrt{n}$) 잡지 않고 **상수로 고정**한 이유는, 블록 하나의 삽입 정렬 비용($O(\text{블록 크기}^2)$)을 입력 전체 크기와 무관하게 유지하기 위해서다. 그래야 이 단계 전체가 $O(n)$ 에 머무른다.

2단계 — 회전 기반 제자리 병합. 정렬된 블록들을 병합 정렬처럼 너비를 두 배씩 늘려가며 합친다. 이때 병합은 추가 배열 없이 두 포인터(`i` , `j`)로 진행한다(`src/sort.c` , `src/sort.py`):

- `a[i] <= a[j]` 이면 왼쪽 원소가 이미 제자리이므로 `i` 만 한 칸 옮긴다.
- `a[i] > a[j]` 이면, `a[i]` 보다 작은 오른쪽 원소들의 구간을 **이분 탐색으로 한 번에** 찾고, 그 구간 전체를 `i` 앞으로 회전(rotate)시켜 옮긴다. 원소를 하나씩 밀어 넣는 대신, "뒤집기 세 번"으로 두 구간을 맞바꾸는 회전 기법을 써서 임시 배열 없이 이동시킨다.

회전을 "한 칸씩 미는 삽입" 대신 "이분 탐색 + 통째로 회전"으로 하는 것이 핵심이다. 한 칸씩 밀면 병합 전체가 $O(n^2)$ 까지 나빠질 수 있지만, 이분 탐색으로 옮길 구간을 한 번에 잡으면 병합 한 번이 $O(n \log n)$ 에 가깝게 유지된다.

- 시간 복잡도: 평균/최악 $O(n \log^2 n)$ (이분 탐색이 낀 회전 병합을 $\log n$ 번의 병합 라운드에 걸쳐 수행하므로, 순수 병합 정렬의 $O(n \log n)$ 보다 $\log n$ 배 정도 더 든다 — $O(1)$ 공간을 얻는 대신 치르는 대가다)
- 공간 복잡도: **$O(1)$** (제자리 정렬 — 병합 정렬과 가장 다른 점)
- 안정 정렬 (`a[i] <= a[j]` 일 때 왼쪽을 먼저 두고, 이분 탐색도 "미만" 기준으로만 오른쪽 원소를 당겨오므로 같은 값의 순서가 바뀌지 않는다)

3. 실험 방법

- **정확성 검증:** 6가지 기본 케이스에 더해, 블록 정렬의 병합 경로(1단계 삽입 정렬만으로 안 끝나고 2단계 회전 병합까지 실제로 실행되는지)를 확인하기 위해 블록 크기(32)보다 큰 200개짜리 배열 케이스를 추가했다. 세 알고리즘 모두에

대해 `make test` 로 확인, 21개 C 체크·8개 Python 테스트 모두 통과, 컴파일 경고 없음.

- 이 케이스를 추가하는 과정에서 처음 구현한 병합 로직(양쪽 구간의 중간점을 서로 이분 탐색하는 분할정복 방식)이 특정 입력에서 **똑같은 호출을 무한히 반복하는 버그**가 있다는 것을 실제로 발견했다. 작은 배열 테스트만으로는 병합 단계 자체가 실행되지 않아 통과했지만, 큰 배열을 넣자 무한 루프로 걸렸다. 그래서 병합 방식을 지금의 "두 포인터 + 이분 탐색 + 회전" 방식으로 다시 짰고, 무작위 배열 2만 개에 대한 별도 검증(파이썬 스크립트)으로 정확성과 안정성을 먼저 확인한 뒤에 코드에 반영했다. 작은 입력만으로는 못 잡는 버그가 있을 수 있다는 것을 배운 부분이라 적어 둔다.
- **성능 측정**: `main.c` / `main.py` 에서 시드를 고정한 난수로 배열을 생성하고, 같은 데이터를 세 알고리즘에 동일하게 입력해 정렬 시간을 측정했다. 측정 후 결과가 실제로 정렬됐는지 다시 확인한다. C는 `clock()` , Python은 `time.perf_counter()` 를 사용했다.
- **입력 크기**: 버블 정렬이 $O(n^2)$ 이라 큰 n 에서 매우 느려지므로, C는 1,000-40,000, Python은 인터프리터 오버헤드를 감안해 100-4,000 범위로 잡았다.
- **실행 환경**: macOS, `gcc -std=c17 -Wall -Wextra -O2` , CPython 3.14. (컨테이너가 아닌 로컬 환경에서 측정했으며, 컨테이너 기준 `make run` 으로 재현 가능하다.)

4. 결과

4.1 C (초 단위)

n	버블 정렬	퀵 정렬	블록 정렬
1,000	0.0007	0.0001	0.0002
5,000	0.0097	0.0002	0.0030
10,000	0.0379	0.0004	0.0057
20,000	0.0870	0.0006	0.0157
40,000	0.3387	0.0013	0.0597

4.2 Python (초 단위)

n	버블 정렬	퀵 정렬	블록 정렬
100	0.0001	0.0000	0.0001
500	0.0029	0.0001	0.0009
1,000	0.0119	0.0003	0.0033
2,000	0.0481	0.0006	0.0124
4,000	0.1978	0.0014	0.0468

5. 분석 및 고찰

버블 정렬은 여전히 압도적으로 느리다. C 기준 $n=40,000$ 에서 0.34초로, 같은 n 에서 퀵 정렬(0.0013초)의 약 260배, 블록 정렬(0.0597초)의 약 6배다. $O(n^2)$ 의 대가를 그대로 보여준다.

퀵 정렬이 셋 중 가장 빠르지만, 블록 정렬은 그 대신 다른 것을 얻는다. C 기준 $n=40,000$ 에서 퀵 정렬이 블록 정렬보다 약 46배 빠르다. 둘 다 비교 기반 정렬이고 평균적으로 $O(n \log n)$ 근방이라는 점은 같지만,

- 퀵 정렬은 추가 메모리를 거의 안 쓰면서($O(\log n)$) 평균적으로 빠르다. 대신 pivot이 나쁘게 걸리면(예: 이미 정렬된 입력에 마지막 원소를 pivot으로 쓰는 경우) 최악 $O(n^2)$ 로 떨어질 수 있다는 위험을 늘 안고 있다.
- 블록 정렬은 $O(1)$ 공간이라는 확실한 이득과, 입력 형태와 무관하게 $O(n \log^2 n)$ 을 대체로 유지한다는 안정성을 얻는 대신, 상수 배가 크다. 병합 한 번마다 이분 탐색과 회전(뒤집기 세 번)을 거치는 비용이 실측 결과에도 그대로 드러난다 — 순수 $O(n \log n)$ 인 병합 정렬(보조 배열을 쓰는 버전)보다 항상 느릴 수밖에 없는 구조다.

두 언어 사이의 격차도 뚜렷하다. 같은 $n=1,000$ 에서 블록 정렬은 C 0.0002초 대비 Python 0.0033초로 약 16배 느리다. 알고리즘의 점근 복잡도는 언어와 무관하지만, 이분 탐색·회전 호출이 반복될 때마다 발생하는 함수 호출 오버헤드가 인터프리터에서 훨씬 크게 누적된다. 버블/퀵 정렬보다 블록 정렬의 C-Python 격차가 상대적으로 더 크게 나타나는 것도 이 때문으로 보인다 — 블록 정렬은 단순 비교·교환이 아니라 "이분 탐색 함수 호출 + 회전 함수 호출"이 반복되는 구조라, 함수 호출 자체가 잦은 Python에서 상수 배가 더 불리하게 작용한다.

공간 복잡도가 진짜 차별점이다. 버블 정렬($O(1)$)과 블록 정렬($O(1)$)은 추가 메모리를 거의 쓰지 않는 제자리 정렬이고, 퀵 정렬은 재귀 스택으로 $O(\log n)$ 을 쓴다. 만약 이번 과제에서 "일반적인" 병합 정렬(보조 배열 $O(n)$)을 같이 구현했다면, 속도는 블록 정렬보다 빠르지만 메모리 사용량에서는 블록 정렬이 확실히 이겼을 것이다 — 이것이 블록 정렬이 실제로 존재하는 이유다.

안정성은 버블 정렬과 블록 정렬이 안정 정렬이고, 퀵 정렬(Lomuto 분할)은 불안정 정렬이다. 정수만 정렬하는 이번 실험에서는 겉으로 드러나지 않지만, 같은 키를 가진 레코드의 원래 순서를 지켜야 하는 실전에서는 중요한 차이다.

6. 결론

- **버블 정렬**: 구현이 가장 단순하지만 $O(n^2)$ 이라 실용적 크기의 입력에서 가장 느리다.
- **퀵 정렬**: 평균적으로 셋 중 가장 빠르고 추가 메모리도 적게 쓰지만, pivot 선택에 따라 최악의 경우 버블 정렬 수준까지 느려질 수 있다는 위험이 코드로도 재현된다.
- **블록 정렬**: 퀵 정렬보다는 느리지만, "추가 배열 없이($O(1)$ 공간) 병합 정렬 수준의 성능을 낸다"는 명확한 목적을 가진 알고리즘이다. 병합 정렬이 배운 알고리즘이었다면, 블록 정렬은 그 병합 정렬의 메모리 단점을 없애기 위해 회전과 이분 탐색이라는 도구를 추가로 들인 결과라고 볼 수 있다.

즉, "가장 빠른 정렬"과 "가장 좋은 정렬"은 다른 질문이다. 속도만 보면 퀵 정렬이 이기지만, **메모리 제약이 있거나 최악의 경우에도 성능이 보장돼야 하거나 안정성이 필요한 상황**에서는 블록 정렬 같은 절충이 의미를 가진다.

7. 참고 자료

- Wikipedia, "Sorting algorithm" — 알고리즘 비교표,
https://en.wikipedia.org/wiki/Sorting_algorithm#Comparison_of_algorithms
- Wikipedia, "Block sort", https://en.wikipedia.org/wiki/Block_sort
- 코드 저장소: <https://github.com/silvpear654/algorithms-hw1>